

Graph Neural Network-Based Soft Actor-Critic for Generalizable Control of Isoperimetric Truss Robots

Student	Isaac Sudweeks
Major	Mechanical Engineering
Faculty Advisor	Dr. Nathan Usevitch
Faculty Reader	Dr. Brady Moon
Honors Coordinator	Dr. Douglas Cook
Project Timeline	Fall 2026 – Winter 2027
Anticipated Completion	April 2027

Working Title

Graph Neural Network-Based Soft Actor-Critic for Generalizable Control of Isoperimetric Truss Robots

1 Project Purpose

For my honors thesis, I propose coupling graph neural networks (GNNs) with a Soft Actor-Critic (SAC) framework so that a single reinforcement learning (RL) model can control many different isoperimetric truss configurations with different graph topologies. Further, this framework will allow a single RL model to generalize to structures that it was not trained on.

2 Project Importance

Isoperimetric truss robots are promising because their modular structure allows many different graph topologies to be assembled from a shared set of components [1]. However, this same modularity creates a difficult control problem: a controller that works for one robot configuration may not transfer to another configuration with a different number of nodes, or edges.

This limitation is especially important for learning-based robotic control. Many reinforcement learning controllers are built around fixed-dimensional multilayer perceptrons (MLPs), which makes it difficult to train a single policy that can operate across structurally different robots. As a result,

modular robots often require separate controllers, retraining, or hand-designed workarounds when their morphology changes.

Some recent work has begun applying GNNs to learned robotic control, but seems to focus primarily on the improved sample efficiency these methods can provide [2], rather than generalizability. This highlights a gap in the literature: whether learning-based robotic control methods such as Soft Actor-Critic can be extended with GNNs in a way that enables successful control on unseen graph structures with potentially different action and observation space dimensionality.

3 Project Overview

3.1 Background and Preliminary Research

An isoperimetric truss robot is built from inflatable tubes that act as structural edges and motorized roller modules that pinch the tubes to create hinge joints. These joints allow the tubes to bend around the roller modules and form shapes such as triangles. The shapes created by the tubes and rollers must maintain a constant perimeter because they are constructed from tubes that preserve their length. Two of the roller modules can be actuated to roll along one of the tubes that forms the triangle; this rolling actuation changes the robot's shape and can induce motion. These triangular units can then be combined into more complex three-dimensional shapes, such as the octahedron shown in Figure 1 [1].

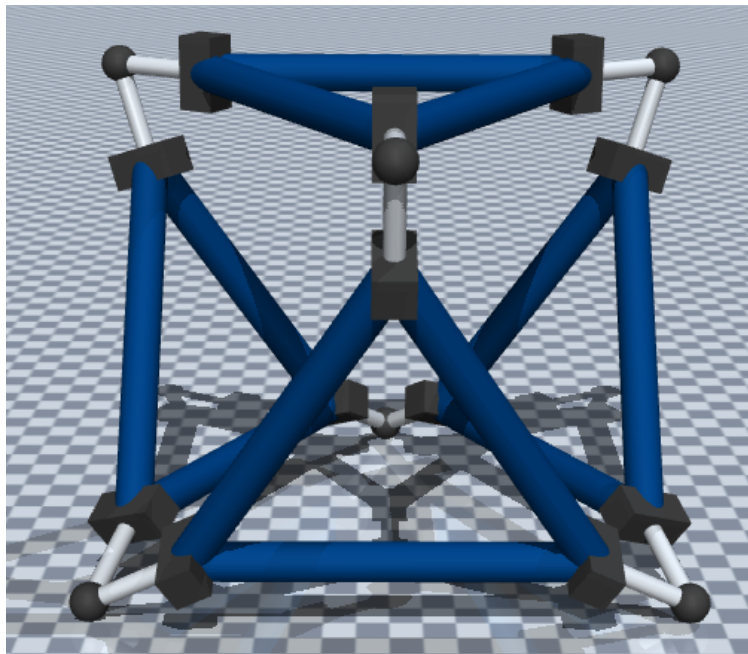


Figure 1: A MuJoCo rendering of an isoperimetric truss robot in an octahedral configuration.

This truss robot construction allows many different shapes and graph topologies to be assembled [3]. In principle, additional units can be added to create a very large number of distinct truss configurations.

Although this highly modular design is one of the benefits of these truss robots, it also presents a rather difficult control problem because traditionally a new control method needs to be learned or tuned for each new truss robot configuration.

In traditional control, an optimization or planning procedure is typically designed to maximize an objective that produces the desired locomotion behavior. These methods can be effective for specific robot morphologies [4], but they are usually tailored to a particular configuration and do not directly provide a single controller that generalizes across many different graph topologies.

Learned robotic control is a different control paradigm in which a policy attempts to learn a mapping from observations to actions that maximize reward over time. Like classical control, this approach can work well, but it has several limitations when applied across multiple morphologies. Most learned robotic control algorithms are based on multilayer perceptrons (MLPs). Because MLP-based controllers usually require fixed-dimensional inputs and outputs, the researcher must know the dimensionality of the robot’s observation and action spaces before training. This inhibits the ability of standard learned control methods to generalize to robot structures with different dimensionalities.

Some researchers have proposed action-space and observation-space masking as a way to address this problem. In this approach, the MLP component of the control algorithm is designed for the largest action and observation spaces that the model is expected to encounter, and unnecessary dimensions are masked out for control tasks that require fewer actions or observations. This approach can work in some cases, but the researcher must still know the largest possible dimensionality in advance, which limits its generalizability [5].

In addition to this problem of generalizability, MLPs generally lack useful inductive bias. MLPs are based on the universal approximation theorem, which states that a neural network with enough parameters can approximate a broad class of functions to arbitrary precision [6]. In practice, however, it is often better to provide neural networks with useful inductive biases so that they can learn the intended behavior more efficiently and reliably [7].

GNNs provide a natural way to impose structure on graph-based systems. Specifically, GNNs assume that the input data can be represented as a graph, with information associated with nodes, edges, or the graph as a whole. This structure allows GNNs to learn from relational data more efficiently and effectively, as has been shown in the literature [8, 7].

In addition to providing useful inductive bias, GNNs can also be viewed as a generalization of convolutional neural networks to irregular graph domains. This makes them especially useful for systems in which the number of nodes or edges may vary between examples because, like CNNs, a single GNN model can be applied across inputs of different sizes, whereas MLPs generally require

fixed-dimensional inputs [8].

3.2 Research Question or Hypothesis

The primary research question of this thesis is whether a deep-learning-based control policy can generalize to truss robot configurations with graph topologies that were not encountered during training. After training, the model will be evaluated on a held-out truss configuration. It will be considered to have generalized successfully if it satisfies the success criteria described below.

3.3 Methodology and Research Plan

A truss robot environment will be developed in MuJoCo in a way that allows the user to generate environments for arbitrary truss robot configurations. The robot will be modeled as a simplified graph structure with nodes and edges corresponding to joints and structural members, respectively, while excluding connector joints and other more realistic geometric details.

3.3.1 Robot Topology Development

Usevitch et al. present algorithms for generating minimally rigid graph structures with varying numbers of nodes and edges, and they enumerate many possible isoperimetric robot graph topologies [3]. These algorithms and enumerated graph topologies will be used to produce eight different isoperimetric truss robot configurations. Of these eight configurations, seven will be used to train the model, and one will be reserved for evaluation as a test of generalization. A Gymnasium environment will be created for each configuration, with the same reward function used across all environments so that the robot is incentivized to accomplish the same task despite differences in truss structure.

3.3.2 Reward Function and Success Criteria

The task targeted in this thesis will be rolling locomotion in the positive x direction. Each environment will use the same reward function and success criteria so that performance can be compared across different truss configurations.

The cumulative reward will be defined as

$$R = \sum_{t=0}^T \gamma^t (c_v v_{\text{COM},x}(t) + c_\lambda C_{\text{rigidity}}(q_t) + c_a - c_E E(t)). \quad (1)$$

Here, $v_{\text{COM},x}(t)$ is the center-of-mass velocity in the positive x direction, C_{rigidity} is the worst case rigidity matrix defined in [4], c_a is the alive bonus, $P_{\text{slip}}(t)$ is the slip penalty, and $E(t)$ is the

actuation-effort penalty. The coefficients c_v , c_λ , c_s , and c_E weight the relative importance of each reward component, and γ is the discount factor.

The worst case rigidity matrix term rewards configurations that remain away from singular or collapsed shapes; in three dimensions, the first six modes correspond to rigid-body motion, so the seventh smallest eigenvalue of the equation $R(x)^T R(x)$ where $R(x)$ is the rigidity matrix provides a proxy for the first non-rigid deformation mode [4, 3]. However, λ_7 scales with graph size which means it cannot reliably be used in a reward function used to compare networks of many different sizes, therefore the worst case rigidity matrix term presented in [4] will be used instead. It is defined as:

$$C_{\text{rigidity}} = \frac{\lambda_7}{\sum_{i=1}^{3n} \lambda_i} = \frac{\lambda_7}{\text{tr}(R(x)^T R(x))} = \frac{\lambda_7}{\sum_{i=1}^{N_L} (L(x)_i)^2} \quad (2)$$

The actuation-effort penalty discourages unnecessarily large control inputs. Because true motor power may not be available directly in every simulation, this term will be approximated using the squared magnitude of the action vector:

$$E(t) = \sum_{j=1}^m u_j(t)^2 = \|u(t)\|_2^2. \quad (3)$$

Here, $u_j(t)$ is the control input applied to actuator j at timestep t , and m is the number of actuated degrees of freedom. If actuator torques and velocities are available in the final simulation, this term may instead be replaced with a mechanical-power estimate.

Each coefficient and threshold will be reported in the thesis. A trial will be considered successful if the robot moves at least 50 units in the positive x direction without entering a failure state. Each robot is built with nodes initialized 1 unit apart, so 50 units shows sustained locomotion rather than accidental or transient motion.

Because a robot structure must stay minimally rigid a robot will have said to enter a failure state if the worst case rigidity matrix falls above some value λ_{crit} [4]:

$$C_{\text{rigidity}} \geq \lambda_{\text{crit}}. \quad (4)$$

3.3.3 Algorithmic Development

A Soft Actor-Critic (SAC) algorithm will be developed following the methods set forth in [9] and [10]. As in standard SAC, the learning framework will contain two main components: an actor and

a critic. The actor will be a stochastic policy trained to maximize the maximum-entropy objective

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))], \quad (5)$$

where $r(s_t, a_t)$ is the task reward, $\mathcal{H}(\pi(\cdot | s_t))$ is the policy entropy at state s_t , and α controls the trade-off between reward maximization and exploration.

The critic will be an ensemble of Q-networks trained to minimize a soft Bellman residual. For critic i , the objective will be written as

$$J_Q(\theta_i) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}, d_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\theta_i}(s_t, a_t) - y_t)^2 \right], \quad (6)$$

where the target value is

$$y_t = r(s_t, a_t) + \gamma(1 - d_t) \mathbb{E}_{a_{t+1} \sim \pi} \left[\min_j Q_{\bar{\theta}_j}(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1} | s_{t+1}) \right]. \quad (7)$$

Here, $\mathcal{D}$ denotes the replay buffer, γ is the discount factor, d_t is a termination indicator that equals one if the episode terminates after timestep t and zero otherwise, and $Q_{\bar{\theta}_j}$ denotes a target critic network. The factor $(1 - d_t)$ removes the bootstrapped future-value term for terminal transitions. Using two critics reduces overestimation bias, while the entropy term keeps the critic target consistent with the maximum-entropy SAC objective.

The actor will be trained to choose actions that have high critic-estimated value while maintaining policy entropy. Its objective will be written as

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_{\phi}} \left[\alpha \log \pi_{\phi}(a_t | s_t) - \min_i Q_{\theta_i}(s_t, a_t) \right]. \quad (8)$$

Here, ϕ denotes the actor parameters. Minimizing this objective encourages the actor to select actions with high predicted Q values while preserving the entropy term used in the maximum-entropy SAC objective.

This framework will be used to develop a baseline SAC algorithm created using standard MLPs as the actor and critic components.

The framework developed in the previous step will then be used to build a custom GNN-SAC algorithm. This algorithm will employ the same overall training setup, but it will use GNNs rather than MLPs for both the actor and the critic networks.

A message passing GNN will be implemented. This type of GNN is mathematically written as the following.

$$h_v^{(k+1)} = \phi^{(k)} \left(h_v^{(k)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}(h_v^{(k)}, h_u^{(k)}, e_{uv}) \right). \quad (9)$$

Here, $h_v^{(k+1)}$ denotes the updated feature vector for node v . The functions $\phi^{(k)}$ and $\psi^{(k)}$ are differentiable functions, which in this work will be implemented as MLPs. The terms $h_v^{(k)}$ and $h_u^{(k)}$ denote the current node features for nodes v and u , respectively. The operator $\oplus$ denotes a differentiable, permutation-invariant aggregation function. Finally, e_{uv} represents optional edge features, when such features are included.

The proposed actor and critic architectures are shown in Figure 2. In both networks, the GNN first maps the graph representation of the robot to a set of node embeddings. For the actor, these node embeddings are interpreted as node-level action outputs. Because the physical actuators correspond to edges rather than isolated nodes, each edge action will be computed by combining the action outputs of the two nodes connected by that edge. This provides a simple way to translate node-level policy outputs into actuator commands while preserving the structure of the robot graph.

The critic will use a similar graph-based representation, but with the selected edge actions included as edge features. After message passing, the resulting node embeddings will be aggregated using a global mean pool to produce a graph-level representation. This pooled representation will then be passed through an MLP to estimate the value of the current state-action pair.

In Figure 2, K denotes the final message-passing layer, d_h denotes the latent embedding dimension, and a_{uv} denotes the action associated with edge (u, v) when it is used as an edge feature in the critic.

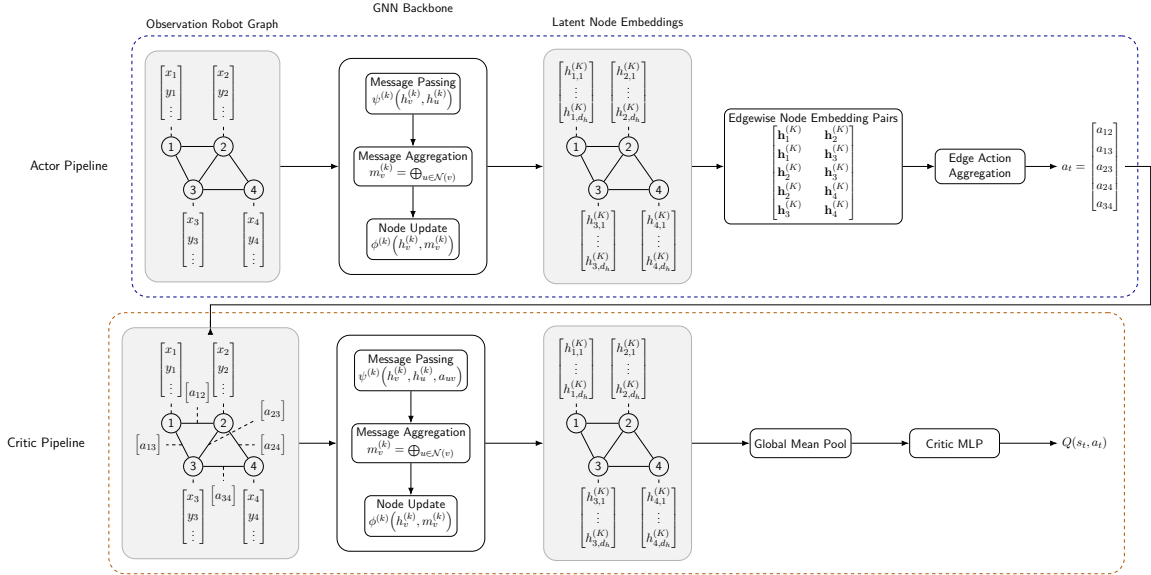


Figure 2: Combined GNN-SAC architecture. In the actor, the observation graph is passed through a message-passing GNN to produce latent node embeddings. Embeddings from each connected node pair are then combined to produce edge-level actions. In the critic, the graph is augmented with these edge actions as edge features and passed through a second message-passing GNN. The resulting latent node embeddings are aggregated with global mean pooling and passed to a critic MLP that estimates the state-action value $Q(s_t, a_t)$.

3.3.4 Entropy Scale and Tuning

One potential issue is that both the policy entropy and the temperature parameter α depend on the scale of the action space. Haarnoja et al. introduce an automatic temperature-tuning method in which α is learned so that the policy entropy is encouraged toward a target value [10]. The loss function is defined as

$$J(\alpha) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} \left[-\alpha \log \pi_\phi(a_t | s_t) - \alpha \bar{\mathcal{H}} \right]. \quad (10)$$

Here, $\bar{\mathcal{H}}$ is the target entropy. Minimizing this objective adjusts α automatically rather than requiring it to be tuned manually.

In standard SAC implementations, the target entropy is often set proportional to the negative dimensionality of the action space. This creates a challenge for GNN-SAC because different robot configurations may have different numbers of actuators and, therefore, different action-space dimensions.

To keep training stable across robot sizes, the entropy term will be normalized by the number of actuators. This prevents larger graphs from inherently producing larger entropy terms. Because the entropy is normalized per actuator, the target entropy will be set to -1 , corresponding to a one-dimensional normalized entropy target.

3.3.5 Comparison and Measure of Generalizability

After the algorithm is developed, a systematic evaluation procedure will be used to determine whether it generalizes to unseen graphs and whether it outperforms the standard SAC baseline.

As described above, the following steps will be performed for both the GNN-SAC and standard SAC algorithms, after which their results will be compared.

1. Hyperparameter search using Bayesian optimization.
2. A 10-million-timestep training run using a random selection of seven different truss configurations with varying numbers of nodes and edges. Action masking will be used to compare the performance of standard SAC to GNN-SAC
3. Evaluation on an unseen graph every 50,000 timesteps, with performance recorded throughout training.

After these steps are completed, the evaluation data for the standard SAC and GNN-SAC algorithms will be compared by plotting reward as a function of training timesteps. The success rate of each model will also be compared.

4 Thesis Committee

1. Faculty Advisor: Dr. Nathan Usevitch
2. Faculty Reader: Dr. Brady Moon
3. Honors Coordinator: Dr. Douglas Cook

5 Qualifications of Thesis Committee

5.1 Faculty Advisor

Name: Dr. Nathan Usevitch

Department: Mechanical Engineering

During his time as a graduate student, Dr. Usevitch helped develop the isoperimetric robot system from the ground up. As a professor, he has continued to develop these systems. As a result, Dr. Usevitch has extensive knowledge of isoperimetric truss robots and their control. As the faculty advisor, he will provide meaningful guidance on the direction and scope of this thesis.

I have worked in Dr. Usevitch's lab for the past six months on learned robotic control through reinforcement learning.

5.2 Faculty Reader

Name: Dr. Brady Moon

Department: Mechanical Engineering

Dr. Brady Moon's research focuses on autonomous robotic systems, motion planning, machine learning for field robotics, and adaptive robotic systems. His expertise is well aligned with this thesis because the project combines reinforcement learning, robotic control, and generalization across robot morphologies. As faculty reader, Dr. Moon will provide valuable feedback on the learning-based control methodology, the experimental design, and the interpretation of the results.

6 Project Timeline

Phase	Activities	Target Date
Phase 1	Submit thesis proposal	May 2026
Phase 2	Complete a focused literature review and finalize specific GNN methodologies	July 2026
Phase 3	Refine simulation environment for GNN use, build base-line MLP-based SAC, and build GNN-based SAC	August 2026
Phase 4	Conduct core experiments, analyze results, and revise methods as needed	October 2026
Phase 5	Complete thesis chapters or sections and circulate drafts to committee members	January 2027
Phase 6	Revise the full thesis, prepare the defense, and submit final materials	February 2027

7 IRB or IACUC Approvals

No IRB or IACUC approval is needed.

8 Funding

Funding will not be requested for this project.

9 Culminating Experience

This work will ideally be shared beyond submission of an honors thesis with the goal of a paper publication at a conference or in a journal. The following table summarizes several plausible publication venues and how well they match the scope of this project.

Venue	Fit	Recommendation	Paper Submission Due Date	Conference Date
CoRL	Extremely high	Best conceptual fit	May	November
ICRA	Extremely high	Best overall target	September	June
RSS	High	Stretch goal	January	July
IROS	High	Safety top-tier	March	September
RA-L	Extremely high	Excellent journal path; can also align with IEEE venues	N/A	N/A

I will also provide access to code created, the data generated, and videos of robot performance.

10 Conclusion

In conclusion, isoperimetric truss robots are promising because they are simple, lightweight, and highly modular. Although that modularity is a major advantage, it also makes control across different robot configurations especially challenging. This work seeks to address that gap by developing a GNN-based SAC algorithm that introduces a stronger inductive bias into the control architecture while also enabling the model to generalize to robot morphologies that were not seen during training.

References

- [1] Nathan S. Usevitch et al. “An untethered isoperimetric soft robot”. In: *Science Robotics* 5.40 (Mar. 2020), eaaz0492. DOI: 10.1126/scirobotics.aaz0492. URL: <https://www.science.org/doi/10.1126/scirobotics.aaz0492> (visited on 05/01/2026).
- [2] Chi Zhang et al. *Morphology-Aware Graph Reinforcement Learning for Tensegrity Robot Locomotion*. Oct. 2025. DOI: 10.48550/arXiv.2510.26067. URL: <http://arxiv.org/abs/2510.26067> (visited on 02/07/2026).
- [3] Nathan Usevitch, Isaac Weaver, and James Usevitch. *Triangle-Decomposable Graphs for Isoperimetric Robots*. May 2025. DOI: 10.48550/arXiv.2505.01624. URL: <http://arxiv.org/abs/2505.01624> (visited on 10/16/2025).
- [4] Nathan S. Usevitch, Zachary M. Hammond, and Mac Schwager. “Locomotion of Linear Actuator Robots Through Kinematic Planning and Nonlinear Optimization”. en. In: *IEEE Transactions on Robotics* 36.5 (Oct. 2020), pp. 1404–1421. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TR0.2020.2995067. URL: <https://ieeexplore.ieee.org/document/9106865/> (visited on 05/02/2026).
- [5] J. K. Terry et al. *Revisiting Parameter Sharing in Multi-Agent Deep Reinforcement Learning*. Oct. 2023. DOI: 10.48550/arXiv.2005.13625. URL: <http://arxiv.org/abs/2005.13625> (visited on 05/14/2026).

- [6] Kurt Hornik. “Approximation capabilities of multilayer feedforward networks”. In: *Neural Networks* 4.2 (Jan. 1991), pp. 251–257. ISSN: 0893-6080. DOI: 10.1016/0893-6080(91)90009-T. URL: <https://www.sciencedirect.com/science/article/pii/089360809190009T> (visited on 05/14/2026).
- [7] Peter W. Battaglia et al. *Relational inductive biases, deep learning, and graph networks*. en. June 2018. URL: <https://arxiv.org/abs/1806.01261v3> (visited on 05/14/2026).
- [8] Jie Zhou et al. “Graph neural networks: A review of methods and applications”. In: *AI Open* 1 (Jan. 2020), pp. 57–81. ISSN: 2666-6510. DOI: 10.1016/j.aiopen.2021.01.001. URL: <https://www.sciencedirect.com/science/article/pii/S2666651021000012> (visited on 05/14/2026).
- [9] Tuomas Haarnoja et al. “Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor”. en. In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR, July 2018, pp. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html> (visited on 02/11/2026).
- [10] Tuomas Haarnoja et al. “Soft Actor-Critic Algorithms and Applications”. en. In: *arXiv preprint* (Dec. 2018). DOI: 10.48550/arXiv.1812.05905. arXiv: 1812.05905 [cs.LG]. URL: <https://arxiv.org/abs/1812.05905>.